

Contents

- Q1. Load the dataset and inspect its structure
- Q2. Confirm the dataset has no missing or duplicate values
- Q3. Separate the independent variables (X) and the dependent variable (y)
- Q4. Split the dataset into training and testing sets (80–20 split)
- Q5. Build and train a Multiple Linear Regression model
- Q6. Predict Price_USD for the test set
- Q7. Print the model's coefficients and intercept; identify the strongest feature
- Q8. Calculate the training score and testing score
- Q9. Compute MAE, MSE, RMSE, and R² for the model's predictions
- Q10. Scatter plot of Actual vs Predicted prices, with a "perfect prediction" reference line
- Conclusion & next steps

Multiple Linear Regression — Car Price Prediction

A worked sample project for machine learning practice

Goal: Predict a used car's selling price (Price_USD) from three numeric features — engine size, mileage, and age — using **Multiple Linear Regression** from scikit-learn.

Dataset: car_price_multiple.csv (sample/synthetic data, 5,000 rows)

Column	Role	Description
EngineSize_L	input	Engine displacement in litres
Mileage_km	input	Total distance driven, in kilometres
CarAge_years	input	Age of the car in years
Price_USD	output	Selling price in US dollars (target variable)

This notebook walks through the full modelling workflow, question by question, with the reasoning behind every step spelled out so it can be read as a standalone tutorial as well as a working notebook.

Q1. Load the dataset and inspect its structure

We start by reading the CSV into a pandas DataFrame, then use `.info()` to see column types and non-null counts, and `.describe()` to get summary statistics (count, mean, std, min, quartiles, max) for every numeric column. This is the standard first step of any ML project — understand what you're working with before touching a model.

In [1]:

```
import pandas as pd

df = pd.read_csv("car_price_multiple.csv")
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype  
---  ---
0   EngineSize_L    5000 non-null   float64
1   Mileage_km      5000 non-null   float64
2   CarAge_years    5000 non-null   int64   
3   Price_USD       5000 non-null   float64
dtypes: float64(3), int64(1)
memory usage: 156.4 KB
```

In [2]:

```
df.describe()
```

	EngineSize_L	Mileage_km	CarAge_years	Price_USD
count	5000.000000	5000.000000	5000.000000	5000.000000
mean	2.987160	98550.930800	7.426800	39386.862108
std	1.159643	56971.919776	4.653447	7542.370645
min	1.000000	525.000000	0.000000	17341.430000
25%	2.000000	49812.000000	3.000000	34186.707500
50%	3.000000	97451.000000	7.000000	39453.035000
75%	4.000000	146800.000000	12.000000	44676.475000
max	5.000000	199885.000000	15.000000	60474.170000

In [3]:

```
df.head()
```

	EngineSize_L	Mileage_km	CarAge_years	Price_USD
0	2.5	79032.0	6	41583.32
1	4.8	94950.0	15	37454.71
2	3.9	170971.0	3	39716.01
3	3.4	68335.0	1	48835.20
4	1.6	173983.0	4	31543.91

Reading the output: all 4 columns hold 5,000 non-null values, matching the row count exactly — a first good sign. Three columns are float64 and one (CarAge_years) is int64. The .describe() table confirms the numeric ranges look sensible (e.g. engine size between 1.0-5.0 L, mileage up to ~200,000 km, no negative prices).

Q2. Confirm the dataset has no missing or duplicate values

Before modelling, we explicitly check for: - **Missing values** with `df.isnull().sum()` — a per-column count of NaNs. - **Duplicate rows** with `df.duplicated().sum()` — an integer count of exact repeated rows.

Skipping this step is a common beginner mistake; a linear regression fit on data with nulls will throw an error, and duplicates can silently bias the model toward whatever rows are over-represented.

In [4]:

```
print(df.isnull().sum())
print("Duplicate Rows:", df.duplicated().sum())
```

```
EngineSize_L    0
Mileage_km      0
CarAge_years    0
Price_USD       0
dtype: int64
Duplicate Rows: 0
```

Result: zero missing values in every column and zero duplicate rows. The dataset is clean and ready to model as-is — no imputation or de-duplication needed.

Q3. Separate the independent variables (X) and the dependent variable (y)

Multiple linear regression needs its inputs and target split apart: - X — a DataFrame of the three predictor columns (EngineSize_L, Mileage_km, CarAge_years). - y — a Series holding the single target column, Price_USD.

In [5]:

```
X = df[['EngineSize_L', 'Mileage_km', 'CarAge_years']]
y = df['Price_USD']
```

Q4. Split the dataset into training and testing sets (80-20 split)

`train_test_split` randomly partitions the data so the model is **trained** on 80% of the rows and **evaluated** on the remaining 20% it has never seen. `random_state=42` fixes the random seed so the split is reproducible — anyone re-running this notebook gets the exact same train/test rows.

In [6]:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
X_train: (4000, 3), X_test: (1000, 3), y_train: (4000,), y_test: (1000,)
```

Result: 4,000 rows go to training and 1,000 to testing — exactly the 80/20 split requested.

Q5. Build and train a Multiple Linear Regression model

`LinearRegression()` from scikit-learn fits the equation

$$\text{Price_USD} = b_0 + b_1 \cdot \text{EngineSize_L} + b_2 \cdot \text{Mileage_km} + b_3 \cdot \text{CarAge_years}$$

by minimizing the sum of squared errors between predicted and actual prices on the training set. Calling `.fit(X_train, y_train)` finds the best-fit coefficients (`b1`, `b2`, `b3`) and intercept (`b0`).

In [7]:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```

Out [7]:

```
LinearRegression()
```

Q6. Predict Price_USD for the test set

With the model trained, `.predict(X_test)` generates a predicted price for every row in the held-out test set. These predictions are what we'll compare against the real prices to judge how well the model generalizes.

```
In [8]: y_pred = model.predict(X_test)
        y_pred[:5]
```

```
Out[8]: [45166.63, 33150.92, 25735.67, 35302.36, 45134.69]
```

Q7. Print the model's coefficients and intercept; identify the strongest feature

Each coefficient tells us how much Price_USD changes for a one-unit increase in that feature, holding the others constant. The intercept is the baseline predicted price when all inputs are zero. Comparing the **absolute value** of the coefficients (via `.abs().idxmax()`) tells us which feature swings the price the most *per unit* — though note this depends on the features' units, so it isn't a perfect measure of "importance" on its own.

```
In [9]: coef = pd.DataFrame({
        'Feature': X.columns,
        'Coefficient': model.coef_.flatten()
    })
        print(coef)
        print("Intercept:", model.intercept_)

        strongest_feature = coef.loc[coef['Coefficient'].abs().idxmax()]
        print("\nStrongest Feature:")
        print(strongest_feature)
```

```
   Feature  Coefficient
EngineSize_L  3189.898653
Mileage_km    -0.090094
CarAge_years -848.726729
Intercept: 45039.577939480514
```

```
Strongest Feature:
Feature      EngineSize_L
Coefficient    3189.898653
Name: 0, dtype: object
```

Interpretation: EngineSize_L has the largest-magnitude coefficient (3189.90), meaning each additional litre of engine size is associated with roughly **\$3190 higher** predicted price, all else equal. CarAge_years has a large *negative* coefficient (cars lose value each year), while Mileage_km has a tiny per-unit coefficient simply because kilometres are a much larger-scale unit than litres or years — its real-world effect over, say, 50,000 km is still substantial.

Q8. Calculate the training score and testing score

`.score(X, y)` returns the model's **R² (coefficient of determination)** — the proportion of variance in Price_USD explained by the model, on a 0-1 scale (1.0 = perfect fit). Comparing the training score to the testing score is a quick check for overfitting: if training R² is much higher than testing R², the model has memorized the training data rather than learning a generalizable pattern.

```
In [10]: train_score = model.score(X_train, y_train)
         test_score = model.score(X_test, y_test)

         print("Training Score:", train_score)
         print("Testing Score:", test_score)
```

```
Training Score: 0.960362
Testing Score: 0.961344
```

Reading the result: training R² (0.960362) and testing R² (0.961344) are almost identical — the testing score is not noticeably lower. That's a strong sign the model generalizes well and is not overfitting to the training data.

Q9. Compute MAE, MSE, RMSE, and R² for the model's predictions

Four complementary error metrics on the test set:

- **MAE** (Mean Absolute Error) — average absolute size of the prediction error, in dollars. Easy to interpret directly.
- **MSE** (Mean Squared Error) — average of *squared* errors; penalizes large errors more heavily than MAE.
- **RMSE** (Root Mean Squared Error) — square root of MSE, bringing the units back to dollars, making it directly comparable to MAE.
- **R²** — same coefficient of determination as Q8, computed here explicitly from the test predictions via `r2_score`.

```
In [11]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE:", round(mae, 2))
print("MSE:", round(mse, 2))
print("RMSE:", round(rmse, 2))
print("R² Score:", round(r2, 4))
```

```
MAE: 1200.21
MSE: 2233186.82
RMSE: 1494.39
R² Score: 0.9613
```

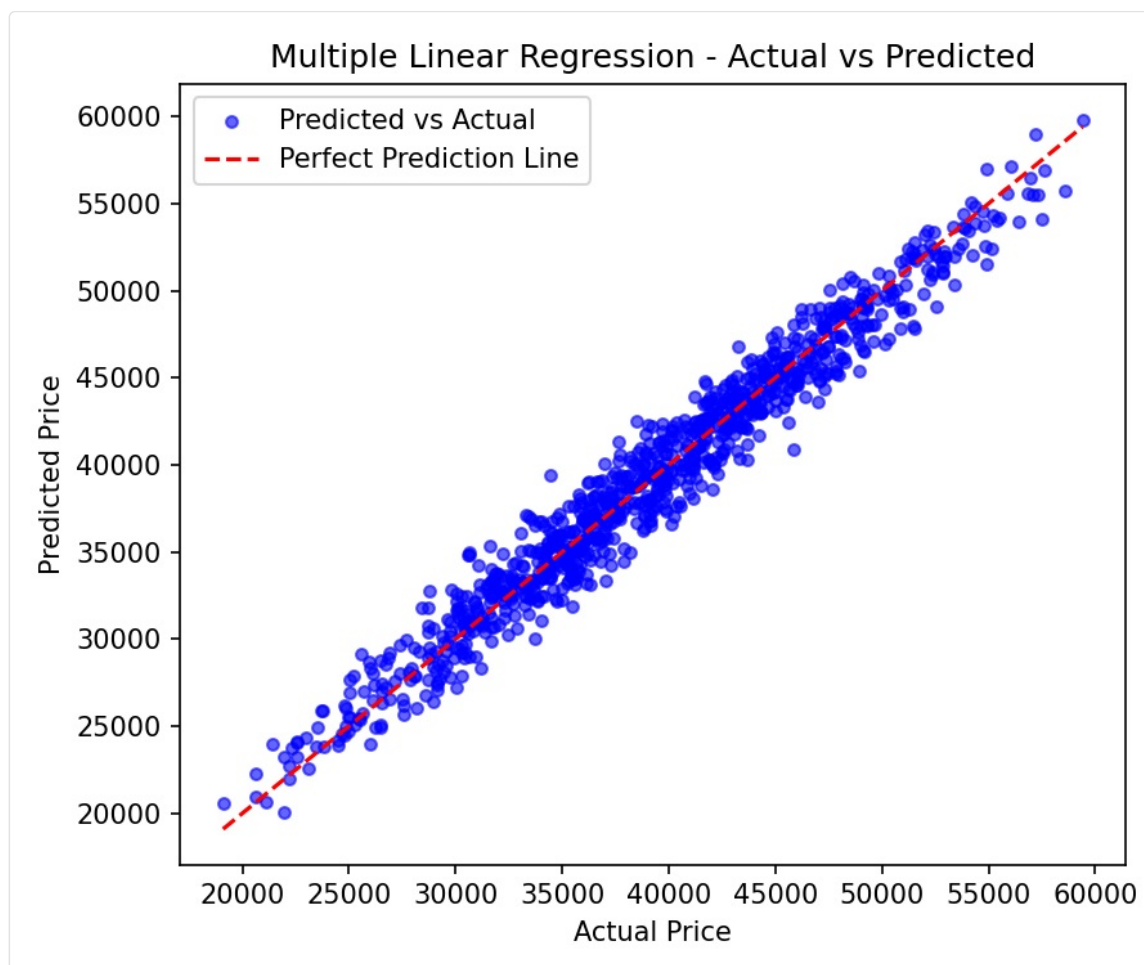
Interpretation: on average the model's price predictions are off by about **\$1,200** (MAE), and the typical error including a penalty for larger misses (RMSE) is about **\$1,494**. With an R^2 of **0.9613**, the three features together explain roughly **96.1%** of the variation in car prices — a strong result for a simple linear model.

Q10. Scatter plot of Actual vs Predicted prices, with a "perfect prediction" reference line

The final check is visual: plot every test-set car's **actual** price on the x-axis against its **predicted** price on the y-axis. A dashed red 45° line marks where actual = predicted exactly — the closer the blue dots cluster around that line, the better the model.

```
In [12]: import matplotlib.pyplot as plt

plt.figure(figsize=(6,5))
plt.scatter(y_test, y_pred, color='blue', label='Predicted vs Actual')
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         color='red', linestyle='--',
         label='Perfect Prediction Line')
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title('Multiple Linear Regression - Actual vs Predicted')
plt.legend()
plt.show()
```



Reading the plot: the points sit tightly around the red reference line across the full price range, with no obvious curve, funnel, or systematic bias — visual confirmation of the $R^2 = 0.9613$ result above. This is what a healthy linear regression fit looks like.

Conclusion & next steps

This notebook covered the complete multiple linear regression workflow: load → clean-check → split features/target → train/test split → fit → predict → interpret coefficients → score → evaluate error → visualize.

Ideas to extend this for a GitHub portfolio piece: - Add **feature scaling** and compare coefficient interpretation. - Try **polynomial features** or **regularization** (Ridge/Lasso) and compare R^2 / RMSE against this baseline. - Use **k-fold cross-validation** instead of a single train/test split for a more robust performance estimate. - Plot **residuals** ($y_{\text{test}} - y_{\text{pred}}$) to check for patterns the scatter plot alone might not reveal.

Generated as a self-contained, executed sample notebook (synthetic dataset, 5,000 rows) for portfolio / GitHub use. All outputs above were produced by actually running the code shown.